Q1 (a) Adam Optimizer & Bias Correction Derivation:

Adam computes 1st moment m_t = beta1 * m_{t-1} + (1 - beta1) * g_t and 2nd moment v_t = beta2 * v_{t-1} + (1

Derivation of Bias Correction:

With m_0 = 0, expanding recursively yields: m_t = (1 - beta1) * sum_{i=1}^t beta1^{t-i} * g_i.

Taking expectations, assuming stationary true expectation E[g_i] = E[g_t]:

E[m_t] = E[g_t] * (1 - beta1) * sum_{i=1}^t beta1^{t-i} = E[g_t] * (1 - beta1) * ((1 - beta1^t)/(1 - beta1)) = E[g_t

Because (1 - beta1^t) < 1, m_t is biased towards 0 during initial steps (especially when beta1 = 0.9).

To obtain an unbiased estimator: m_hat_t = m_t / (1 - beta1^t) such that E[m_hat_t] = E[g_t].

Similarly for second moment: v_hat_t = v_t / (1 - beta2^t) ensures E[v_hat_t] = E[g_t^2].

Q1 (b) SGD+NAG vs Adam in Non-Convex Curvatures:

Standard SGD oscillates heavily across high-curvature ravine walls with slow progress down the valley.

Nesterov Accelerated Gradient (NAG) calculates gradients at theta + beta*v (look-ahead step),

which dampens oscillations before committing the update, but still relies on a single scalar learning rate.

Adam dynamically scales learning rates per coordinate by dividing by sqrt(v_hat_t) + eps.

This suppresses steep gradients with high second moments and accelerates flat dimensions with low variance,

enabling rapid traversal of ravines and quick escape from saddle points.

Q2 (a) Scaled Dot-Product Attention & Variance Derivation:

Attention formula: Attention(Q, K, V) = softmax((Q * K^T) / sqrt(d_k)) * V.

Mathematical Proof of Variance Scaling:

Assume components of q and k are independent with mean E[q_i] = 0 and variance Var(q_i) = 1.

The dot product is: q . k = sum_{i=1}^{d_k} q_i * k_i.

E[q . k] = sum E[q_i] * E[k_i] = 0.

Var(q . k) = sum Var(q_i * k_i) = sum (Var(q_i) * Var(k_i)) = d_k * (1 * 1) = d_k.

Standard deviation equals sqrt(d_k). For large d_k, dot products grow huge in magnitude,

pushing softmax into extreme saturating regions where its derivative d(softmax)/dz -> 0 (vanishing gradients).

Dividing by sqrt(d_k) scales variance back: Var((q . k)/sqrt(d_k)) = Var(q . k)/d_k = d_k/d_k = 1, ensuring stable gradie

Q2 (b) Multi-Head Attention Representation & Complexity:

Multi-Head Attention projects Q, K, V into h distinct lower-dimensional subspaces of dimension d_k = d_model / h.

This allows the network to jointly attend to different representation spaces (e.g. syntactic, positional, coreference).

Single-head attention averages over these diverse subspaces, destroying directional specialization.

Complexity: Self-attention requires O(n^2 * d) operations with O(1) sequential path length.

RNNs require O(n * d^2) operations with O(n) sequential path length, precluding parallelization across tokens.

Q3 (a) BatchNorm vs LayerNorm:

BatchNorm computes mean/variance across batch dimension N for each feature channel.
It fails when batch size is small (N < 16) and requires running population statistics during inference.
LayerNorm computes mean/variance across all feature channels for each sample independently.
It has zero dependence on batch size and works seamlessly for variable-length sequential Transformer models.

Q3 (b) Inverted Dropout Expectation Derivation:

Inverted Dropout scales activations during training: a_train = (mask * a) / (1 - p), where mask ~ Bernoulli(1 - p).
Expectation during training: E[a_train] = E[(mask * a)/(1 - p)] = ((1 - p)*a)/(1 - p) = a.
Since training expectation equals unmasked activation a, NO scaling is required at test time (a_test = a)!

Q4 (a) ResNet Gradient Flow Derivation & Vanishing Gradient Prevention:

Residual formulation: x_{l+1} = x_l + F(x_l). Telescoping to layer L: x_L = x_l + sum_{i=l}^{L-1} F(x_i).
Gradient with respect to x_l: dE/dx_l = (dE/dx_L) * (dx_L/dx_l) = (dE/dx_L) * [1 + d/dx_l sum_{i=l}^{L-1} F(x_i)].
The additive identity term 1 guarantees that even if the weight Jacobian d/dx_l sum F vanishes to 0,
the gradient dE/dx_l retains the direct signal (dE/dx_L) * 1, preventing vanishing gradients in 100+ layer networks.
Q4 (b) Pre-Activation ResNets: Moves BN and ReLU inside residual branch F(x), keeping the skip path purely linear: x_{l+1} = x_l + F(ReLU(BN(x_l))), which unlocks seamless gradient propagation.